

Хорошо, я подготовлю документ с описанием модели данных на основе наших обсуждений.

Модель Данных Системы Финансового Планирования

Этот документ описывает структуру базы данных, предназначенную для системы личного финансового планирования. Модель ориентирована на гибкость, простоту управления для пользователя и поддержку осознанного принятия финансовых решений.

1. Сущности Базы Данных

1.1. **User** (Пользователь)

- **Назначение:** Идентификация и аутентификация пользователя системы.
- **Поля:**
 - **id:** **SERIAL** (Primary Key) – Уникальный идентификатор пользователя.
 - **username:** **VARCHAR(255)** (Unique, Not Null) – Имя пользователя для входа.
 - **email:** **VARCHAR(255)** (Unique, Not Null) – Электронная почта пользователя.
 - **password_hash:** **VARCHAR(255)** (Not Null) – Хеш пароля.
 - **created_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время создания записи.
 - **updated_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время последнего обновления.

1.2. **Account** (Счет/Кошелек)

- **Назначение:** Представление реальных мест хранения денежных средств пользователя (например, банковский счет, наличные, кредитная карта).
- **Поля:**
 - **id:** **SERIAL** (Primary Key) – Уникальный идентификатор счета.
 - **user_id:** **INTEGER** (Foreign Key to **User.id**, Not Null) – Идентификатор пользователя-владельца.
 - **name:** **VARCHAR(255)** (Not Null) – Наименование счета (например, "Карта Сбера", "Наличные USD").
 - **type:** **VARCHAR(50)** (Not Null) – Тип счета (например, 'bank_account', 'cash', 'credit_card', 'e_wallet', 'loan_account', 'investment_account').
 - **currency_code:** **VARCHAR(3)** (Not Null) – Трехбуквенный код валюты (например, 'RUB', 'USD', 'EUR').
 - **opening_balance:** **DECIMAL(19, 4)** (Default: 0.00) – Начальный баланс счета при его заведении в систему (если он не нулевой).
 - **notes:** **TEXT** (Nullable) – Заметки пользователя по счету.
 - **is_archived:** **BOOLEAN** (Default: **false**) – Флаг, указывающий, что счет архивирован и не используется активно.
 - **created_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время создания.
 - **updated_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время обновления.
- **Примечание:** Текущий баланс счета вычисляется на основе **opening_balance** и всех **Transaction** по этому счету, либо по данным последней **InventoryEntry**.

1.3. **InventoryEntry** (Запись Инвентаризации)

- **Назначение:** Фиксация фактического баланса **Account** на определенную дату, введенная пользователем для сверки.
- **Поля:**
 - **id:** **SERIAL** (Primary Key) – Уникальный идентификатор записи.
 - **account_id:** **INTEGER** (Foreign Key to **Account.id**, Not Null) – Идентификатор счета.
 - **entry_date:** **DATE** (Not Null) – Дата, на которую зафиксирован баланс.
 - **reported_balance:** **DECIMAL(19, 4)** (Not Null) – Заявленный пользователем баланс.
 - **created_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время создания.
- **Использование:** Для Вьюхи 5 (Инвентаризация и соответствие баланса кошельков).

1.4. **Transaction** (Транзакция)

- **Назначение:** Запись о фактическом движении денежных средств (доход или расход).
- **Поля:**
 - **id:** **SERIAL** (Primary Key) – Уникальный идентификатор транзакции.
 - **account_id:** **INTEGER** (Foreign Key to **Account.id**, Not Null) – Идентификатор счета, по которому прошла транзакция.
 - **user_id:** **INTEGER** (Foreign Key to **User.id**, Not Null) – Идентификатор пользователя.
 - **description:** **VARCHAR(255)** (Nullable) – Описание транзакции.
 - **amount:** **DECIMAL(19, 4)** (Not Null) – Сумма транзакции. Положительная для дохода, отрицательная для расхода.
 - **currency_code:** **VARCHAR(3)** (Not Null) – Код валюты транзакции. Должен совпадать с **Account.currency_code** для простоты, либо требуется механизм конвертации.
 - **transaction_date:** **TIMESTAMP** (Not Null) – Фактическая дата и время совершения транзакции.
 - **category_id:** **INTEGER** (Foreign Key to **Category.id**, Nullable) – Идентификатор категории транзакции.
 - **is_irregular:** **BOOLEAN** (Default: **false**) – Флаг для отметки нерегулярных доходов/расходов (для Вьюхи 14).
 - **notes:** **TEXT** (Nullable) – Дополнительные заметки.
 - **created_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время добавления записи в систему.
 - **updated_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время обновления.

1.5. **Category** (Категория)

- **Назначение:** Пользовательская классификация **Transaction** для анализа и отчетности.
- **Поля:**
 - **id:** **SERIAL** (Primary Key) – Уникальный идентификатор категории.
 - **user_id:** **INTEGER** (Foreign Key to **User.id**, Not Null) – Идентификатор пользователя.
 - **name:** **VARCHAR(255)** (Not Null) – Наименование категории.
 - **type:** **VARCHAR(50)** (Not Null, e.g., 'income', 'expense') – Тип категории (доходная/расходная).
 - **parent_category_id:** **INTEGER** (Foreign Key to **Category.id**, Nullable) – Для иерархической структуры категорий.
 - **created_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время создания.
 - **updated_at:** **TIMESTAMP** (Default: **CURRENT_TIMESTAMP**) – Дата и время обновления.

1.6. Plan (План)

- **Назначение:** Концептуальный "контейнер" для денег, представляющий цель, бюджет, фонд и т.д. Ядро слоя планирования.
- **Поля:**
 - **id:** SERIAL (Primary Key) – Уникальный идентификатор плана.
 - **user_id:** INTEGER (Foreign Key to User.id, Not Null) – Идентификатор пользователя.
 - **name:** VARCHAR(255) (Not Null) – Наименование плана.
 - **type:** VARCHAR(50) (Not Null) – Тип плана, определяющий его поведение:
 - **goal:** Цель накопления (например, "Новый телефон").
 - **fund:** Фонд (например, "Резервный фонд").
 - **budget_category:** Бюджетная категория на период (например, "Продукты на месяц").
 - **planned_major_expense:** Планируемый крупный разовый расход (например, "Первоначальный взнос").
 - **transitory:** (Предложение) Транзитный план, средства в котором не задерживаются (для Вьюхи 2).
 - **target_amount:** DECIMAL(19, 4) (Nullable) – Целевая сумма для типов **goal**, **planned_major_expense**.
 - **target_date:** DATE (Nullable) – Целевая дата для типов **goal**, **planned_major_expense**.
 - **parent_plan_id:** INTEGER (Foreign Key to Plan.id, Nullable) – Для иерархии планов.
 - **status:** VARCHAR(50) (Not Null, Default: 'active') – Статус плана ('active', 'achieved', 'on_hold', 'archived', 'cancelled').
 - **priority:** INTEGER (Default: 0) – Приоритет для **DistributionRule**. Выше значение = выше приоритет.
 - **notes:** TEXT (Nullable) – Заметки пользователя.
 - **importance_level:** VARCHAR(50) (Default: 'normal') – Важность плана как точки контроля ('normal', 'constraint', 'critical') (для Вьюхи "Планы как ограничения").
 - **is_auto_spend_all:** BOOLEAN (Default: false) – Альтернатива/дополнение к типу **transitory**. Если true, план исключается из некоторых представлений (Вьюха 2).
 - **created_at:** TIMESTAMP (Default: CURRENT_TIMESTAMP) – Дата и время создания.
 - **updated_at:** TIMESTAMP (Default: CURRENT_TIMESTAMP) – Дата и время обновления.

1.7. ForecastEntry (Прогнозная Запись / Поток)

- **Назначение:** Описание ожидаемых (будущих или повторяющихся) денежных потоков – доходов или расходов. Основа для прогнозирования.
- **Поля:**
 - **id:** SERIAL (Primary Key) – Уникальный идентификатор.
 - **user_id:** INTEGER (Foreign Key to User.id, Not Null) – Идентификатор пользователя.
 - **description:** VARCHAR(255) (Not Null) – Описание потока (например, "Зарплата", "Аренда квартиры").
 - **amount_by_scenario:** JSONB (Not Null) – Суммы для различных сценариев. Пример: {"baseline": 100.00, "optimistic": 120.00, "pessimistic": 80.00}. Сумма положительна для дохода, отрицательна для расхода.
 - **currency_code:** VARCHAR(3) (Not Null) – Код валюты.
 - **type:** VARCHAR(50) (Not Null, 'income', 'expense') – Тип потока.

- **recurrence_rule**: VARCHAR(255) (Nullable) – Правило повторения (например, RRULE строка, 'once', 'daily', 'weekly', 'monthly_on_day_X'). Если **null**, событие разовое на **start_date**.
- **start_date**: DATE (Not Null) – Дата начала действия потока или дата разового события.
- **end_date**: DATE (Nullable) – Дата окончания для повторяющихся потоков.
- **linked_plan_id**: INTEGER (Foreign Key to **Plan.id**, Nullable) – План, с которым напрямую связан этот расход (уменьшает его прогнозный баланс) или к которому относится этот доход (информационно, т.к. доходы сначала попадают в "общий котел").
- **scenario_tags**: TEXT[] (Nullable) – Массив тегов сценариев, к которым относится эта запись. Если **null** или пустой, относится ко всем сценариям. Позволяет включать/исключать запись из определенных сценариев.
- **notes**: TEXT (Nullable) – Заметки.
- **created_at**: TIMESTAMP (Default: **CURRENT_TIMESTAMP**) – Дата и время создания.
- **updated_at**: TIMESTAMP (Default: **CURRENT_TIMESTAMP**) – Дата и время обновления.

1.8. DistributionRule (Правило Распределения / Трансфер)

- **Назначение:** Описание правил перемещения *виртуальных* денежных средств: между планами или из общего пула прогнозируемых доходов в планы.
- **Поля:**
 - **id**: SERIAL (Primary Key) – Уникальный идентификатор.
 - **user_id**: INTEGER (Foreign Key to **User.id**, Not Null) – Идентификатор пользователя.
 - **description**: VARCHAR(255) (Nullable) – Описание правила.
 - **source_plan_id**: INTEGER (Foreign Key to **Plan.id**, Nullable) – Исходный план (только для **type** = 'plan_to_plan_transfer' или **reconciliation_transfer**). Если **null**, источник – общий пул доходов.
 - **destination_plan_id**: INTEGER (Foreign Key to **Plan.id**, Not Null) – Целевой план.
 - **amount_by_scenario**: JSONB (Nullable) – Суммы перевода для различных сценариев. Используется, если правило переводит фиксированную сумму.
 - **percentage_of_source**: JSONB (Nullable) – Процент от источника для различных сценариев (например, от **ForecastEntry** типа **income**). Пример: {"baseline": 10.00, "optimistic": 15.00} (означает 10% и 15%). Используется один из: **amount_by_scenario** или **percentage_of_source**.
 - **linked_forecast_entry_id**: INTEGER (Foreign Key to **ForecastEntry.id**, Nullable) – Если правило срабатывает по факту определенного **ForecastEntry** (например, распределение зарплаты).
 - **type**: VARCHAR(50) (Not Null) – Тип правила:
 - **income_distribution_to_plan**: Распределение из общего дохода в план.
 - **plan_to_plan_transfer**: Перевод между планами.
 - **reconciliation_transfer**: Специальный перевод для сверки расхождений.
 - **recurrence_rule**: VARCHAR(255) (Nullable) – Правило повторения.
 - **effective_date_condition**: DATE (Nullable) – Дата, с которой правило начинает действовать.
 - **priority**: INTEGER (Default: 0) – Приоритет выполнения (важно, если ресурсов на все правила не хватает).
 - **scenario_tags**: TEXT[] (Nullable) – Теги сценариев.
 - **is_active**: BOOLEAN (Default: **true**) – Активно ли правило.
 - **created_at**: TIMESTAMP (Default: **CURRENT_TIMESTAMP**) – Дата и время создания.

- `updated_at`: `TIMESTAMP` (Default: `CURRENT_TIMESTAMP`) – Дата и время обновления.

1.9. `Scenario` (Сценарий Прогноза)

- **Назначение:** Именованный набор предположений для прогнозирования. Позволяет моделировать различные финансовые ситуации.
- **Поля:**
 - `id`: `SERIAL` (Primary Key) – Уникальный идентификатор.
 - `user_id`: `INTEGER` (Foreign Key to `User.id`, Not Null) – Идентификатор пользователя.
 - `name`: `VARCHAR(255)` (Not Null, Unique per user) – Имя сценария (например, "Базовый", "Оптимистичный").
 - `description`: `TEXT` (Nullable) – Описание сценария.
 - `is_baseline`: `BOOLEAN` (Default: `false`) – Является ли этот сценарий базовым/основным.
 - `created_at`: `TIMESTAMP` (Default: `CURRENT_TIMESTAMP`) – Дата и время создания.
 - `updated_at`: `TIMESTAMP` (Default: `CURRENT_TIMESTAMP`) – Дата и время обновления.

1.10. `TransactionPlanLink` (Связь Транзакции с Планом)

- **Назначение:** Опционально связывает фактическую `Transaction` с одним или несколькими `Plan`, указывая, как реальные деньги были отнесены к планам.
- **Поля:**
 - `id`: `SERIAL` (Primary Key) – Уникальный идентификатор.
 - `transaction_id`: `INTEGER` (Foreign Key to `Transaction.id`, Not Null) – Идентификатор транзакции.
 - `plan_id`: `INTEGER` (Foreign Key to `Plan.id`, Not Null) – Идентификатор плана.
 - `amount_linked`: `DECIMAL(19, 4)` (Not Null) – Какая часть суммы транзакции относится к этому плану.
 - `notes`: `TEXT` (Nullable) – Заметки.
 - `created_at`: `TIMESTAMP` (Default: `CURRENT_TIMESTAMP`) – Дата и время создания.
- **Примечание:** Сумма `amount_linked` по всем связям для одной транзакции не должна превышать общую сумму транзакции.

1.11. `ForecastSnapshot` (Снимок Прогноза) - *Опционально*

- **Назначение:** Хранение исторических версий прогнозов для сравнения (Вьюха 3).
- **Поля:**
 - `id`: `SERIAL` (Primary Key).
 - `user_id`: `INTEGER` (Foreign Key to `User.id`, Not Null).
 - `scenario_id`: `INTEGER` (Foreign Key to `Scenario.id`, Nullable) – Сценарий, на основе которого сделан снимок.
 - `snapshot_name`: `VARCHAR(255)` (Not Null) – Имя снимка (например, "Прогноз на Июль 2024 (создан 01.07.2024)").
 - `snapshot_date`: `DATE` (Not Null) – Дата создания снимка.
 - `forecast_data`: `JSONB` (Not Null) – Данные прогноза (например, массив объектов `{plan_id, date, projected_balance}`).
 - `created_at`: `TIMESTAMP` (Default: `CURRENT_TIMESTAMP`).

- **Вариант:** Вместо хранения полных данных, можно хранить только ключевые параметры `ForecastEntry` и `DistributionRule`, которые были активны на момент снимка, и пересчитывать прогноз "на лету". Это сложнее, но экономит место.

2. Вычисляемые Значения и Логика

2.1. `Account.CurrentBalance` (Текущий Баланс Счета)

- **Расчет:**
 1. Если есть `InventoryEntry` для счета: баланс последней `InventoryEntry.reported_balance` + сумма `Transaction.amount` по этому счету, где `Transaction.transaction_date > InventoryEntry.entry_date`.
 2. Если нет `InventoryEntry`: `Account.opening_balance` + сумма всех `Transaction.amount` по этому счету.
- **Назначение:** Отображение актуального баланса реального счета.

2.2. `Plan.CurrentActualAllocatedAmount` (Текущая Фактически Выделенная Сумма на План)

- **Расчет (на текущую дату):** `SUM(TransactionPlanLink.amount_linked WHERE Transaction.amount > 0 AND TransactionPlanLink.plan_id = Plan.id)` (приходные фактические транзакции, связанные с планом) - `SUM(TransactionPlanLink.amount_linked WHERE Transaction.amount < 0 AND TransactionPlanLink.plan_id = Plan.id)` (расходные фактические транзакции, связанные с планом) + `SUM(DistributionRule.amount_by_scenario.baseline WHERE type = 'reconciliation_transfer' AND destination_plan_id = Plan.id AND rule_executed_factually)` - `SUM(DistributionRule.amount_by_scenario.baseline WHERE type = 'reconciliation_transfer' AND source_plan_id = Plan.id AND rule_executed_factually)` (плюс/минус другие фактически исполненные `DistributionRule`, переводящие реальные средства)
- **Назначение:** Показывает, сколько реальных денег пользователя на данный момент концептуально находится "внутри" этого плана. Используется для Выходов 1, 2 и как база для будущих прогнозов.

2.3. `Plan.ProjectedException(target_date, scenario_tag)` (Прогнозный Баланс Плана)

- **Расчет:**
 1. **Начальная точка:** `Plan.CurrentActualAllocatedAmount` на `current_date` (сегодня).
 2. **Применение потоков:** Хронологически, с `current_date` до `target_date`, применяются все релевантные `ForecastEntry` (соответствующие `scenario_tag` или общие, с учетом `amount_by_scenario`) и все релевантные `DistributionRule` (также с учетом сценария и `amount_by_scenario/percentage_of_source`).
 - Доходы (`ForecastEntry.type='income'`) увеличивают общий виртуальный пул или, если `linked_plan_id` указан, могут напрямую влиять на этот план.
 - Расходы (`ForecastEntry.type='expense'`) с `linked_plan_id = Plan.id` уменьшают баланс плана.
 - `DistributionRule` типа `income_distribution_to_plan` (из общего пула в план) или `plan_to_plan_transfer` (из/в план) изменяют баланс плана.

- **Назначение:** Основа для большинства прогнозных выходов (3, 8, 9, 10, 12, 16, 17, 18).

2.4. **Plan.CalculatedDiscrepancy** (Расчетное Расхождение по Плану)

- **Расчет (на текущую дату для базового сценария):**

`ProjectedBalanceBasedOnOriginalPlanForDate(current_date, 'baseline') - Plan.CurrentActualAllocatedAmount`

- `ProjectedBalanceBasedOnOriginalPlanForDate`: Это гипотетический расчет `ProjectedBalance` на `current_date`, как если бы он начинался с нуля (или с точки последней "идеальной" сверки) и учитывал только `ForecastEntry` и `DistributionRule`, которые должны были произойти до `current_date`, без учета фактических `TransactionPlanLink` или `reconciliation_transfer`.
- **Назначение:** Показывает "over/under" (перерасход/недотрату) по плану. Основа для решения пользователя о необходимости сверки.
- **Иерархия:** Значения агрегируются вверх по `parent_plan_id`. Родительский план показывает сумму расхождений своих дочерних планов плюс собственное расхождение.

2.5. **UnallocatedActualFunds** (Нераспределенные Фактические Средства)

- **Расчет:** `SUM(Account.CurrentBalance для всех Account) - SUM(Plan.CurrentActualAllocatedAmount для всех планов верхнего уровня)`
- **Назначение:** Показывает, какая часть реальных денег пользователя еще не отнесена ни к одному плану.

2.6. **NetWorthForecast(target_date, scenario_tag)** (Прогноз Чистого Капитала)

- **Расчет:**
 1. **Начальная точка:** `SUM(Account.CurrentBalance для всех Account)` на `current_date`.
 2. **Применение будущих потоков:** С `current_date` до `target_date`:
 - `+ SUM(ForecastEntry.amount_by_scenario[scenario_tag] WHERE type = 'income' AND scenario matches)`
 - `- SUM(ForecastEntry.amount_by_scenario[scenario_tag] WHERE type = 'expense' AND scenario matches)`
- **Назначение:** Этот расчет не должен учитывать внутренние переводы между планами (`DistributionRule` типа `plan_to_plan_transfer` между накопительными планами), так как они не меняют общий капитал.
- **Назначение:** Оценка общего финансового состояния в будущем.

3. Управление Данными и Жизненный Цикл

- **Создание и Ведение Планов/Прогнозов:** Пользователь создает `Plan`, затем определяет будущие денежные потоки через `ForecastEntry` и правила их распределения по планам через `DistributionRule`. Сценарии (`Scenario`) позволяют моделировать различные варианты.
- **Учет Факта:** Ежедневные `Transaction` регистрируются и опционально связываются с планами через `TransactionPlanLink`.
- **Сверка (Reconciliation):**
 1. Система рассчитывает `Plan.CalculatedDiscrepancy`.
 2. Пользователь видит расхождения.

3. Для коррекции пользователь создает `DistributionRule` типа `reconciliation_transfer` (или обычный `plan_to_plan_transfer`), чтобы переместить виртуальные средства и обнулить расхождение на выбранном уровне плана. Этот трансфер влияет на `Plan.CurrentActualAllocatedAmount`.

- **Эволюция Оценок:** Пользователь изменяет `amount_by_scenario` в `ForecastEntry` или `DistributionRule` по мере уточнения информации, "сужая воронку" прогнозов.
- **Архивация:** Старые или неактуальные `Plan`, `Account`, `Scenario` могут быть помечены как архивированные (`is_archived` или `status='archived'`).

4. Покрытие Требований и Выходов

Эта модель данных обеспечивает основу для всех 18 описанных выходов:

- **Прогнозные выходы (1, 3, 8, 9, 10, 12, 16, 17, 18):** Основаны на `Plan.ProjectedException`, `ForecastEntry`, `DistributionRule` и `Scenario`.
- **Фактические/аналитические выходы (2, 4, 5, 7, 11, 13, 14, 15):** Основаны на `Transaction`, `Account`, `InventoryEntry`, `Plan.CurrentActualAllocatedAmount`, `Plan.CalculatedDiscrepancy`, `TransactionPlanLink`, `Category`.
- **Сравнение прогнозов (Выход 3):** Требуется либо реализации `ForecastSnapshot`, либо продвинутой логики работы со `Scenario` (например, возможность "заморозить" сценарий).
- **Время задержки средств (Выход 6):** В текущей модели напрямую не отслеживается "путь каждой денежной единицы". Может быть реализовано упрощенно (например, анализ среднего времени нахождения денег на определенных типах планов) или потребует усложнения модели (что противоречит требованию простоты). *Это остается областью для дальнейшего рассмотрения или упрощения требований.*

5. Простота Управления для Пользователя

- **Основные операции:** Создание планов, ввод транзакций, настройка повторяющихся доходов/расходов (`ForecastEntry`) и правил распределения (`DistributionRule`) должны быть интуитивно понятны.
- **Сверка:** Упрощена до стандартных операций перевода.
- **Сценарии:** Позволяют гибко моделировать будущее без изменения основного плана.
- **Визуализация:** Ключевую роль играет UI, который должен наглядно представлять расчетные показатели (балансы, прогнозы, расхождения).

6. Возможные Непокрытые Аспекты и Варианты

- **История изменения прогнозов (детальная):** Если требуется детальная история того, как менялся каждый параметр прогноза, может понадобиться версионирование `ForecastEntry` и `DistributionRule` или более сложная система снимков. Текущая модель с `ForecastSnapshot` предлагает компромисс.
- **Точное отслеживание "времени жизни" каждой денежной единицы (Выход 6):** Как отмечено, это сложно без усложнения.
- **Производительность расчетов:** Для сложных сценариев с большим количеством планов и длительным горизонтом прогнозирования, расчеты "на лету" могут быть медленными. Потребуется кэширование или периодический предварительный расчет агрегированных данных (например, ночной расчет `PlanBalanceSnapshot`).

Этот документ представляет собой основу. Реализация потребует тщательной проработки UI/UX для обеспечения простоты использования и детального тестирования расчетной логики.